

信息安全基础综合设计实验

Lecture 09

李经纬

电子科技大学

课程回顾

对称密码

➤流密码：RC4

- 使用伪随机数生成器替代一次一密随机密钥
- 处理单位：比特 → 低错误传播

➤分组密码：DES

- 处理单位：分组 → 扩散
- 基于Feistel网络 → 加解密具有相似性
- 分组密码应用模式：ECB、CBC、CTR → 处理多个分组长度数据

哈希函数

➤特性：单向、抗碰撞

➤实例：SHA1

- 160比特（20字节）长度哈希值

GDB基础操作

为什么需要GDB

➤传统调试方法的局限

- printf调试：修改源码→重新编译→运行→查看输出
- 静态分析：只能发现语法错误和简单逻辑问题
- 代码审查：依赖人工经验，容易**遗漏**复杂问题

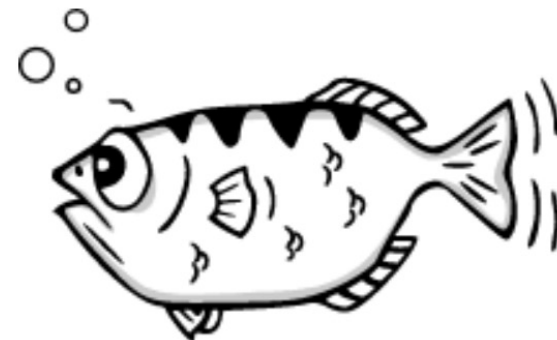
➤GDB的优势：

- 实时检查：**无需修改**源码即可查看程序状态
- 动态分析：运行时检查变量值和内存状态
- 精确控制：单步执行，精确定位问题发生位置

GDB简介

➤ GNU Debugger (GDB)

- 支持C、C++、Go、Rust等多种编程语言
- 开源免费，在Linux/Unix环境下广泛使用



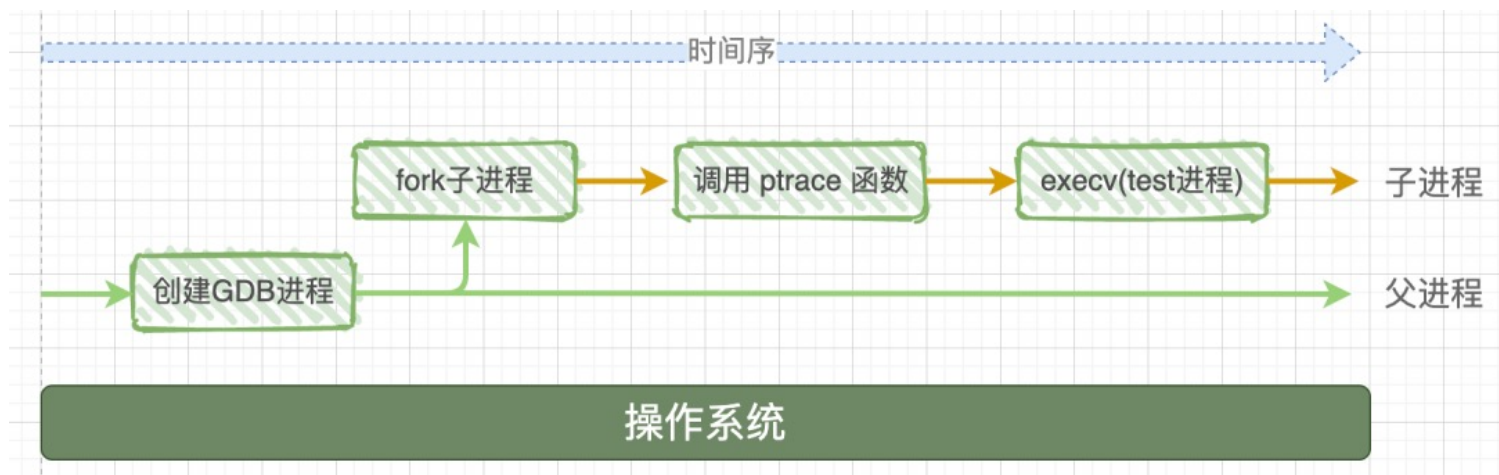
➤ 核心功能：

- 程序执行控制：暂停、继续、单步执行，也可以在**任意位置**设置断点
- 状态检查：查看变量、内存、寄存器内容
- 调用栈分析：追踪函数调用层次关系
- 段错误调试：快速定位内存访问**错误**，追踪函数调用层次关系

GDB基本原理

➤ 进程间调试机制

- GDB作为父进程，通过fork创建子进程调试环境
- 使用ptrace系统调用实现进程间通信
- 目标程序作为子进程运行，**完全**受GDB控制



编译调试版可执行文件

➤需要特殊编译选项

- 默认编译: `gcc program.c -o program`
- 调试编译: `gcc -g -O0 -Wall program.c -o program`
- 区别: **调试版本**包含额外信息, 便于GDB分析

➤调试编译选项详解:

- `-g`: 包含调试信息 (符号表、行号映射)
- `-O0`: **关闭**编译优化, 保证调试信息**准确性**
- `-Wall`: 显示所有警告信息
- `-o program`: 指定输出文件名

启动GDB

➤版本控制基础：

```
gdb # 启动GDB
gdb program # 启动时加载可执行文件
gdb --args program arg1 arg2 # 启动时设置可执行文件的命令行参数
```

➤基本界面：

- (gdb) 提示符：等待用户输入命令
- Tab键：命令自动补全
- Enter键：**重复**执行上一条命令
- Ctrl+C：**中断**当前操作
- Ctrl+D：快捷键退出, q：正常退出

加载并运行程序

➤加载程序：

- 加载可执行文件：`file program`，也可以启动时直接加载程序`gdb program`
- **注意：**`program`**必须**是用`-g`选项编译的程序
- 加载后GDB**读取**调试信息和符号表

➤运行程序：

<code>run</code>	# 不带参数运行
<code>run arg1 arg2</code>	# 带输入参数运行
<code>run < input.txt</code>	# 重定向标准输入
<code>run > output.txt</code>	# 重定向标准输出

断点设置 I

➤什么是断点

- 程序执行过程中的预设停止点
- 允许在特定位置检查程序状态
- **不修改**程序源码或可执行文件 (**不同于**传统printf调试)

➤基本断点设置:

<code>break main</code>	# 在main函数入口设置断点
<code>break 15</code>	# 在当前文件第15行设置断点
<code>break crash.c:22</code>	# 在指定文件的指定行设置断点
<code>break print_message</code>	# 在函数入口设置断点

断点设置 II

➤删除断点

```
delete          # 删除所有断点
delete 2        # 删除编号为2的断点
clear main      # 删除main函数上的断点
clear crash.c:22 # 删除指定文件第22行上的断点
```

➤断点控制:

```
disable 2      # 禁用编号为2的断点
enable 2       # 启用编号为2的断点
ignore 2 5     # 忽略断点2的前5次触发
info breakpoints # 显示所有断点的详细信息
```

断点设置 III

➤什么是条件断点

- 只有**满足**特定条件时才会触发的断点
- 避免在**不必要**的位置频繁停止，是更符合调试需求的断点

➤设置条件断点：

```
break crash.c:10 if i == 50      # 设置时指定条件
break crash.c:10                 # 先设置断点
condition 1 i == 50              # 后添加条件
```

➤条件表达式：

- 支持C语言的比较操作符：`==`，`!=`，`<`，`>`，`<=`，`>=`和逻辑操作符：`&&`，`||`，`!`
- 可以使用变量名和函数调用

单步执行控制

➤基本执行命令

<code>continue (c)</code>	# 继续执行到下一个断点
<code>next (n)</code>	# 执行下一行（不进入函数内部）
<code>step (s)</code>	# 执行下一行（进入函数内部）
<code>finish</code>	# 执行完当前函数并返回

➤命令区别说明：

- `next`：将函数调用视为一个整体，一次函数调用算一行
- `step`：进入函数内部逐行执行
- `finish`：快速**执行完**当前函数（配合`step`使用）

➤循环控制：

- `until`，执行直到循环**结束**
- `until 20`，执行到第20行

变量和表达式查看 I

➤基本变量查看

```
print var          # 打印变量值
print *ptr         # 打印指针指向的值
print arr[5]       # 打印数组元素
```

➤表达式计算:

```
print i + j        # 计算表达式
print func(arg)     # 调用函数并打印返回值
print sizeof(int)   # 打印类型大小
```

➤数组查看:

```
print arr[0]@10     # 打印数组前10个元素
print *arr@10       # 等价写法
```


变量和表达式查看 II

➤变量地址和类型查看

<code>ptype var</code>	# 显示详细类型信息
<code>info locals</code>	# 显示当前函数的所有局部变量
<code>info args</code>	# 显示当前函数的参数

➤指针分析:

<code>print ptr</code>	# 指针值 (地址)
<code>print *ptr</code>	# 指针指向的内容
<code>print &ptr</code>	# 指针本身的地址

➤结构体查看:

<code>ptype struct student</code>	# 显示结构体定义
<code>print student</code>	# 打印整个结构体
<code>print student.name</code>	# 打印结构体成员

源码显示和导航

➤源码显示:

```
list                # 显示当前位置附近源码
list 15             # 显示第15行附近源码
list main           # 显示main函数开始处源码
list crash.c:20     # 显示指定文件的指定行
```

➤源码导航:

```
list -              # 显示之前的源码
list +              # 显示之后的源码
set listsize 20     # 设置每次显示20行
```

调用栈

➤调用栈概念

- 记录函数调用层次的数据结构
- 每个函数调用创建一个**栈帧 (Stack Frame)**

➤调用栈命令:

<code>backtrace (bt)</code>	# 显示完整调用栈
<code>backtrace 5</code>	# 只显示最近5层调用
<code>frame 2</code>	# 切换到第2号栈帧

➤栈帧信息:

<code>info frame</code>	# 显示当前栈帧详细信息
<code>info registers</code>	# 显示寄存器内容
<code>info stack</code>	# 显示栈的使用情况

监视点使用

➤什么是监视点

- 监视变量值变化的特殊断点，用于跟踪变量的修改时机和位置
- 断点的目标是**代码行**，监视点的目标是**变量(或内存地址)**，按需使用

➤监视点类型：

```
watch var          # 变量被写入时停止
rwatch var         # 变量被读取时停止
awatch var         # 变量被访问时停止
```

➤监视点管理：

```
info watchpoints   # 显示所有监视点
delete watchpoint 1 # 删除监视点
```

GDB实战示例

示例程序（包含bug）

➤ **Lecture 01** 作业: `convert.cpp` (**省略**用于测试的主函数)

```
// 二进制转十进制 (有bug版本)
unsigned int
binary_to_decimal(string input)
{
    unsigned int result = 0;
    int len = input.length();

    for(int i = 0; i < len; i++)
    {
        result = result * 2;
        if(input[i] == '1') {
            result = result + 1;
        }
    }
    return result;
}
```

```
// 十进制转二进制 (有bug版本)
string decimal_to_binary(unsigned
int input) {
    if(input == 0) return "0";

    string result = "";
    while(input > 0) {
        if(input % 2 == 1) {
            result = "1" + result;
        } else {
            result = "0" + result;
        }
        input = input / 2;
    }
    return result;
}
```

实战调试 I

➤步骤1：编译并运行

```
• zihasyu@intel4:~/WZY/Debug$ g++ -g -O0 -Wall convert.cpp -o convert  
• zihasyu@intel4:~/WZY/Debug$ ./convert
```

➤输出结果：

```
Binary 10a10 = Decimal 10 # 错误！应该报错或返回0，所以第二次调用有  
bug
```

```
Binary 1010 = Decimal 10  
Binary 10a10 = Decimal 18  
Decimal 10 = Binary 1010  
Decimal 255 = Binary 11111111
```

实战调试 II

➤步骤2：发现问题并启动GDB

```
zihasyu@intel4:~/WZY/Debug$ gdb convert
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
```

➤步骤3：在可疑函数设置断点

```
(gdb) break binary_to_decimal
Breakpoint 1 at 0x5555555563c9: file convert.cpp, line 6.
(gdb) run
Starting program: /home/zihasyu/WZY/Debug/convert

Breakpoint 1, binary_to_decimal (
    input=<error: Cannot access memory at address 0x5de0894c08c48348>)
at convert.cpp:6
```


实战调试 III

➤步骤4：单步调试第一次函数调用（无bug的输入）

- `$2 len` 显示的是**垃圾值**，因为第8行还未执行完，变量已声明但**未初始化**
- `$3 = 4`，现在第8行已执行完毕，len 被**正确赋值**为4。

```
(gdb) next
7         unsigned int result = 0;
(gdb) next
8         int len = input.length();
(gdb) print input
$1 = "1010"
(gdb) print len
$2 = 32767
(gdb) next
10        for(int i = 0; i < len; i++) {
(gdb) print len
$3 = 4
(gdb) continue
Continuing.
Binary 1010 = Decimal 10

Breakpoint 1, binary_to_decimal (
    input=<error: Cannot access memory at address 0x5de0894c08c48348>) at convert.cpp:6
6         unsigned int binary_to_decimal(const string& input) {
```

实战调试 IV

➤ 步骤5：单步调试第二次函数调用（有bug的输入）

- `$4 = "10a10"`：包含**非法**字符的输入
- `(gdb) break 12`：在循环体设置**断点**

```
(gdb) next
7           unsigned int result = 0;
(gdb) next
8           int len = input.length();
(gdb) print input
$4 = "10a10"
(gdb) print input.length()
$5 = 5
(gdb) break 12
Breakpoint 2 at 0x555555556401: file convert.cpp, line 12.
(gdb) continue
Continuing.

Breakpoint 2, binary_to_decimal (input="10a10") at convert.cpp:12
12           if(input[i] == '1') {
```

实战调试 V

➤步骤6：检查第一次循环 (i=0)

- input[0] 是字符 '1' , ASCII码是 49, result 当前为 0 (进入的状态)
- 一切正常

```
Breakpoint 2, binary_to_decimal (input="10a10") at convert.cpp:12
12             if(input[i] == '1') {
(gdb) print i
$6 = 0
(gdb) print input[i]
$7 = (const __gnu_cxx::__alloc_traits<std::allocator<char>, char>::value_type &) @0x7fffffff
d940: 49 '1'
(gdb) print (char)input[i]
$8 = 49 '1'
(gdb) print result
$9 = 0
(gdb) continue
Continuing.
```

实战调试 VI

➤步骤7：检查第二次循环 (i=1)

- input[1] 是字符 '0' , ASCII码是 48, result 当前为 2
- 一切正常

```
Breakpoint 2, binary_to_decimal (input="10a10") at convert.cpp:12
12             if(input[i] == '1') {
(gdb) print i
$10 = 1
(gdb) print (char)input[i]
$11 = 48 '0'
(gdb) print result
$12 = 2
(gdb) continue
Continuing.
```

实战调试 VII

➤步骤8：检查第三次循环（i=2） - 发现问题

- input[2] 是字符 'a', ASCII码 97, result 当前为 4
- 由于 `'a' != '1'`, 不会进入if, 程序把 'a' 当作 '0' 处理, 没有报错

```
Breakpoint 2, binary_to_decimal (input="10a10") at convert.cpp:12
12             if(input[i] == '1') {
(gdb) print i
$13 = 2
(gdb) print (char)input[i]
$14 = 97 'a'
(gdb) print result
$15 = 4
(gdb) continue
Continuing.
```

实战调试 VIII

➤ 步骤9：检查第四次循环（i=3） - 锁定问题

- input[3] 是字符 '1', ASCII码 19, result 当前为 8
- 说明上一次循环中确实将'a'当作 '\0' 处理

```
Breakpoint 2, binary_to_decimal (input="10a10") at convert.cpp:12
12         if(input[i] == '1') {
(gdb) print i
$16 = 3
(gdb) print (char)input[i]
$17 = 49 '1'
(gdb) print result
$18 = 8
(gdb) continue
Continuing.
```

其他常用调试方案

➤查看调用栈

```
(gdb) backtrace
#0  binary_to_decimal (input="10a10") at convert.cpp:11
#1  0x... in main () at convert.cpp:35
```

➤使用监视点观察变量变化：每次 result 改变时都会停止

```
(gdb) watch result
Hardware watchpoint 3: result
(gdb) continue
```

➤修改变量测试修复方案：

```
# 在遇到'a'时，手动设置result为0测试效果
(gdb) set variable result = 0
(gdb) print result
$9 = 0
```

常用信号处理

➤ 常见信号类型

- SIGSEGV: 段错误, **非法**内存访问
- SIGFPE: 浮点异常, 除零错误
- SIGABRT: 程序主动调用abort()
- SIGINT: 用户**中断** (Ctrl+C)

➤ 信号处理命令:

```
info signals          # 显示所有信号处理方式
handle SIGSEGV stop   # 遇到段错误时停止程序
handle SIGINT nostop  # 忽略中断信号
handle SIGFPE print   # 打印浮点异常信息
```


常见错误类型与调试方法 I

➤ 段错误 (Segmentation Fault)

- 原因：访问**未分配或无权限**的内存
- 调试：backtrace定位，print检查指针值
- 预防：指针使用前检查NULL，数组边界检查

➤ 空指针引用：

- 现象：对NULL指针进行**解引用**操作
- 调试：print检查指针值是否为0x0
- 预防：指针赋值后立即检查有效性

常见错误类型与调试方法 II

➤数组越界

- 原因：访问数组定义**范围外**的内存
- 调试：使用条件断点检查数组索引
- 预防：循环边界检查，使用安全的访问函数

➤内存泄漏：

- 原因：动态分配内存后未正确**释放**
- 调试：watch监视malloc/free配对

GDB使用技巧汇总

➤ 命令简化技巧

- 大部分命令支持首字母缩写: r(run), b(break), c(continue)
- 回车键**重复**上一条命令
- Tab键自动补全命令和参数

➤ 调试策略:

- 二分法: 在问题可能存在的区域中间设置断点
- 自顶向下: 从main函数开始, 逐层深入
- 自底向上: 从**崩溃点**开始, 向上追溯调用栈

➤ 日志记录:

```
set logging on           # 开启日志记录
set logging file debug.log # 所有GDB输出将保存到debug.log
```

附：GDB常用命令快速参考

GDB命令	描述
<code>file <program></code>	加载程序
<code>run <arg1> <arg2></code>	运行程序
<code>break <breakpoint></code>	设置断点
<code>continue</code>	继续执行
<code>next</code>	下一行(不进入函数)
<code>Step</code>	下一行(进入函数)
<code>print <var></code>	打印变量/表达式
<code>Bt</code>	显示调用栈
<code>List</code>	显示源码
<code>watch <var></code>	设置监视点
<code>info <breakpoint></code>	显示信息
<code>delete <breakpoint></code>	删除断点
<code>q</code> 或 <code>ctrl+D</code>	退出GDB

自学资源

➤ GDB Tutorial:

https://www.cs.toronto.edu/~krueger/csc209h/tut/gdb_tutorial.html

➤ GDB官方手册:

<https://www.gnu.org/software/gdb/documentation/>

课堂作业

作业要求

➤作业内容：

➤评分等级：

- **课堂上**完成作业，正确解释代码，通过助教登记，获得平时成绩加分
- 课后按时（本周日24点之前）完成作业并发送至邮箱
3216086385@qq.com，获得本次作业平时成绩
 - 邮件题目：【周X】姓名+学号
- 未完成作业，无法获得本次作业平时成绩